PAD-aula10 - Observer | Prof. Marlon

Disciplina: Padrões de Projeto | **Categoria:** GoF (Comportamental) | **Linguagem:** Java

---

Objetivos da Aula

- Compreender o padrão de projeto Observer e sua aplicabilidade.
- Identificar cenários de dependência um-para-muitos.
- Implementar a comunicação desacoplada entre objetos.
- Analisar o impacto do padrão na manutenção e escalabilidade do software.

Motivação e Contextualização

No desenvolvimento de sistemas, frequentemente nos deparamos com situações onde o estado de um objeto muda e outros objetos dependentes precisam ser notificados dessa alteração. Em uma abordagem ingênua, o objeto "observado" teria que conhecer cada um dos objetos "observadores" para disparar métodos individualmente. Isso gera um acoplamento forte, dificultando a adição de novos componentes sem modificar o código existente. O padrão Observer resolve isso estabelecendo um contrato de assinatura.

Exemplo Problemático (Sem o Padrão)

Considere um sistema de notícias onde uma classe `AgenciaNoticias` envia mensagens diretamente para `EmailSubscriber` e `SMSsubscriber`.

```
public class AgenciaNoticias { private EmailSubscriber emailSub = new EmailSubscriber(); private SMSSubscriber smsSub = new SMSSubscriber(); public void novaNoticia(String msg) { emailSub.receber(msg); smsSub.receber(msg); } }
```

Análise do Problema: Este código viola o princípio Aberto/Fechado (SOLID). Para cada novo tipo de assinante (ex: Notificação Push), precisaríamos alterar a classe `AgenciaNoticias`, adicionando um novo campo e uma nova chamada de método. O acoplamento é estático e rígido.

Solução com o Padrão Observer

O padrão Observer define uma interface `Observer` e uma interface `Subject`. O Subject mantém uma lista de Observers e notifica todos quando necessário.

```
public interface Observer { void atualizar(String msg); } public class AgenciaNoticias { private List<Observer> assinantes = new ArrayList<>(); public void
```

```
adicionar(Observer o) { assinantes.add(o); } public void notificar(String msg) {
for(Observer o : assinantes) o.atualizar(msg); } }
```

Análise da Solução: Agora, `AgenciaNoticias` não conhece as classes concretas de seus assinantes, apenas a interface `Observer`. Podemos adicionar infinitos novos tipos de assinantes sem alterar uma única linha de código na classe do Subject.

Sem o Padrão x Com o Padrão

Critério	Sem Observer	Com Observer
Acoplamento	Forte (Concreto)	Fraco (Abstrato)
Extensibilidade	Difícil	Alta (Plug & Play)

Vantagens e Desvantagens

Vantagens
- Desacoplamento total.
- Suporte a broadcasts.
- Princípio Aberto/Fechado.

Desvantagens
- Possibilidade de memory leaks.
- Ordem de notificação não garantida.
- Complexidade na depuração.

Exercícios de Fixação

Exercício 1:
Implemente um sistema onde um objeto `EstacaoMeteorologica` notifica `DisplayCelular` e `DisplayWeb` sobre a temperatura atual.

Exercício 2:
Desafio: Como você removeria um observador específico de uma lista durante a notificação sem gerar um `ConcurrentModificationException`?

Será que compreendemos o padrão? Então responda:

1. Por que o Observer é classificado como comportamental?
2. É possível que um Objeto seja ao mesmo tempo Subject e Observer? Explique.
3. Como o padrão lida com a remoção de observadores que não existem mais?

Fechamento Sintetizado

- O Observer é a base para sistemas orientados a eventos.
- Promove a inversão de dependência através de interfaces.
- Indispensável para interfaces gráficas e sistemas de mensageria.